

DESIGN & IMPLEMENTATION OF PID BASED DC MOTOR SPEED CONTROL USING ESP32 & L298N MOTOR DRIVE

PROJECT REPORT

2026

A closed-loop hardware system designed to precisely regulate DC motor speed using a PID controller. Features an ESP32 microcontroller and a custom Python GUI for real-time telemetry plotting and dynamic over-the-air parameter tuning via TCP/WiFi.

PRESENTED BY :

Linuka Jaysinghe



1. Introduction.....	2
2. Design Methodology	2
2.1 System Architecture	2
2.2 Mathematical Model of PID Controller.....	3
2.3 Role of Each PID Component.....	3
2.4 System Block Diagram.....	3
2.5 Circuit Design	3
3. Component Selection	4
3.1 ESP32 Microcontroller	4
3.2 L298N Motor Driver	4
3.3 Encoder Motor.....	4
3.4 Feedback Mechanism	4
4. Implementation	5
4.1 PWM-Based Speed Control	5
4.2 PID Algorithm Execution.....	5
4.3 Software Implementation (ESP32 & GUI).....	6
4.4 Sample Calculations	6
5. Results and Analysis	8
5.1 System Response Characteristics.....	8
5.2 PID Tuning Results	8
5.3 Performance Observation	8
5.4 Speed vs Time Graph	8
6. Discussion	9
7. Challenges Faced.....	10
8. Improvements	10
9. Conclusion	10
10. References.....	10
Appendix A:	11
Appendix B:	15

1. Introduction

In modern control engineering, precise regulation of dynamic systems is essential for achieving efficiency, stability, and reliability. DC motor speed control is a fundamental problem widely encountered in industrial automation, robotics, and embedded systems.

This project focuses on the design and implementation of a closed-loop DC motor speed control system using a Proportional-Integral-Derivative (PID) controller. The system is implemented using an ESP32 microcontroller and an L298N motor driver module.

Unlike open-loop systems, which cannot compensate for disturbances or parameter variations, the closed-loop PID controller continuously monitors system output and adjusts control input accordingly. This enables accurate tracking of the desired speed (setpoint) while minimizing error.

The primary objective of this project is to:

- Design a real-time PID control system
- Implement PWM-based motor speed control
- Analyze system performance using control theory concepts
- Evaluate system stability and response characteristics

2. Design Methodology

2.1 System Architecture

The overall system operates as a feedback control loop consisting of:

- Reference input (desired speed)
- PID controller
- ESP32 (control unit)
- L298N motor driver
- Encoder Motor
- Feedback sensor (Quadrature Encoder measuring RPM)

The controller continuously calculates the error between the desired and actual speed and adjusts the PWM signal accordingly.

2.2 Mathematical Model of PID Controller

The control signal generated by the PID controller is defined as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Where:

- $e(t) = r(t) - y(t)$ is the error signal
- $r(t)$ = reference input (setpoint)
- $y(t)$ = measured output (motor speed)

2.3 Role of Each PID Component

- **Proportional (Kp):**
Produces output proportional to the current error. Increasing K_p reduces rise time but may introduce oscillations.
- **Integral (Ki):**
Eliminates steady-state error by accumulating past errors. However, excessive K_i can lead to instability.
- **Derivative (Kd):**
Predicts future error by considering rate of change. It improves damping and reduces overshoot.

2.4 System Block Diagram

Python GUI (Setpoint/Tuning) → WiFi (TCP) → ESP32 (PID Controller & PWM) → L298N Motor Driver → DC Motor → Encoder Feedback → ESP32 Interrupts

2.5 Circuit Design

The ESP32 generates PWM signals through its GPIO pins using the 'ledcWrite' function, which is connected to the enable (ENB) pin of the L298N module. Motor direction is handled by the L298N, and the motor's encoder outputs (Phase A and Phase B) are connected to the ESP32's input pins with internal pull-ups enabled to read the rotational pulses. An external power supply is used to drive the motor, while a common ground ensures proper signal reference.

3. Component Selection

3.1 ESP32 Microcontroller

The ESP32 was selected due to:

- High processing speed (dual-core processor)
- Built-in PWM functionality
- Native WiFi capabilities for wireless telemetry and control.
- Hardware interrupt support for accurate encoder reading.

3.2 L298N Motor Driver

The L298N module is an H-bridge driver that allows:

- Bidirectional motor control
- High current handling
- Isolation between control and power circuits

3.3 Encoder Motor

This system uses a DC motor with a square encoder. The encoder has a resolution of 300 pulses per revolution (PPR). This hardware allows for highly accurate, real-time measurement of the motor's rotational speed, which is essential for tight closed-loop control.

3.4 Feedback Mechanism

In this implementation, feedback is obtained via the physical motor encoder. The ESP32 uses a hardware interrupt that is triggered on the rising edge of the A-channel of the encoder. By checking the state of the B-channel during this interrupt, the direction of rotation is determined, and the pulse count is updated accordingly. Accurate feedback is critical for effective closed-loop control.

4. Implementation

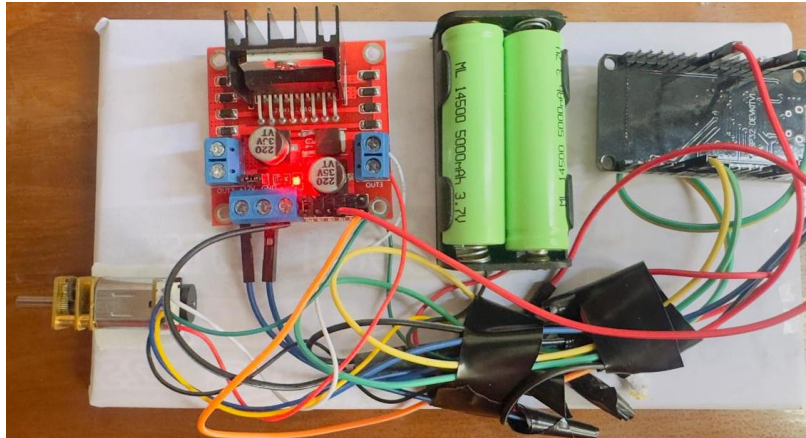


Figure 1: Fully Assembled ESP32 and L298N Closed-Loop Motor Control Prototype.

4.1 PWM-Based Speed Control

PWM (Pulse Width Modulation) is used to control motor speed by adjusting duty cycle:

$$\text{Average Voltage} \propto \text{Duty Cycle}$$

Higher duty cycle results in higher motor speed.

4.2 PID Algorithm Execution

The control loop executes continuously based on a defined sampling time interval:

1. Measure current speed (calculate RPM from encoder delta)
2. Compute error
3. Apply PID equation (calculating Proportional, Integral, and Derivative terms over time dt)
4. Constrain the Integral sum to prevent integral windup.
5. Update PWM output (constrained to the allowable MIN/MAX PWM values)
6. Repeat in real-time

4.3 Software Implementation (ESP32 & GUI)

The embedded code uses a WiFiServer to open a TCP socket on port 8080. This server continuously listens for a client connection. At the same time, a Python Tkinter application connects to this socket. The ESP32 logs Setpoint, Measured RPM, and PWM data to the Python client at regular intervals. The Python GUI parses this stream and dynamically updates a Matplotlib graph, giving the user a live visual representation of the system's response. The user can send new target RPM, K_p, K_i, and K_d values from the GUI, which the ESP32 receives, parses, and immediately applies to the active control loop.

The complete C++ source code for the ESP32 microcontroller and the Python script for the GUI can be found in **Appendix A** and **Appendix B**, respectively.

4.4 Sample Calculations

To better understand the operation of the PID controller, a sample calculation is performed for one iteration of the control loop.

Given:

- Setpoint speed = 250
- Measured speed = 200
- $K_p = 2.5$, $K_i = 1.5$, $K_d = 0.03$
- Previous error = 40
- Accumulated integral = 50
- Time step (dt) = 0.1 seconds

Step 1: Calculate Error

$$e(t) = \text{Setpoint} - \text{Measured Value} = 250 - 200 = 50$$

Step 2: Update Integral Term

$$\text{Integral} = 50 + (50 * 0.1) = 55$$

Step 3: Calculate Derivative Term

$$\text{Derivative} = (e(t) - \text{Previous Error})/dt = (50 - 40)/0.1 = 100$$

Step 4: Compute PID Output

$$\begin{aligned}\text{Output} &= (K_p \times e(t)) + (K_i \times \text{Integral}) + (K_d \times \text{Derivative}) \\ &= (2.5 \times 50) + (1.5 \times 55) + (0.03 \times 100) \\ &= 125 + 82.5 + 3 \\ &= 210.5\end{aligned}$$

Step 5: Apply Output Limiting

Since PWM values must be within the range 0–255:

$$\text{Final PWM Output} = 210$$

Interpretation:

The calculated PWM value of 210 is applied to the motor driver, increasing the motor speed to reduce the error. This process repeats continuously, allowing the system to reach and maintain the desired speed.

5. Results and Analysis

5.1 System Response Characteristics

The performance of the system is evaluated using:

- Rise Time (T_r): Time taken to reach desired speed
- Overshoot (M_p): Maximum deviation beyond setpoint
- Settling Time (T_s): Time to stabilize within tolerance
- Steady-State Error (E_{ss}): Final error after stabilization

5.2 PID Tuning Results

Parameter	Value
K_p	2.5
K_i	1.5
K_d	0.03

5.3 Performance Observation

- The system achieved stable operation within a short time.
- Overshoot was minimized using derivative control ($K_d = 0.03$).
- Integral action eliminated steady-state error, aided by integral windup constraints (-300.0 to 300.0).
- Overall response was smooth and controlled, directly visible in real-time via the Python matplotlib plotting canvas.

5.4 Speed vs Time Graph

Using the Python application, the expected curve actively maps:

- Rapid rise
- Slight overshoot
- Smooth settling

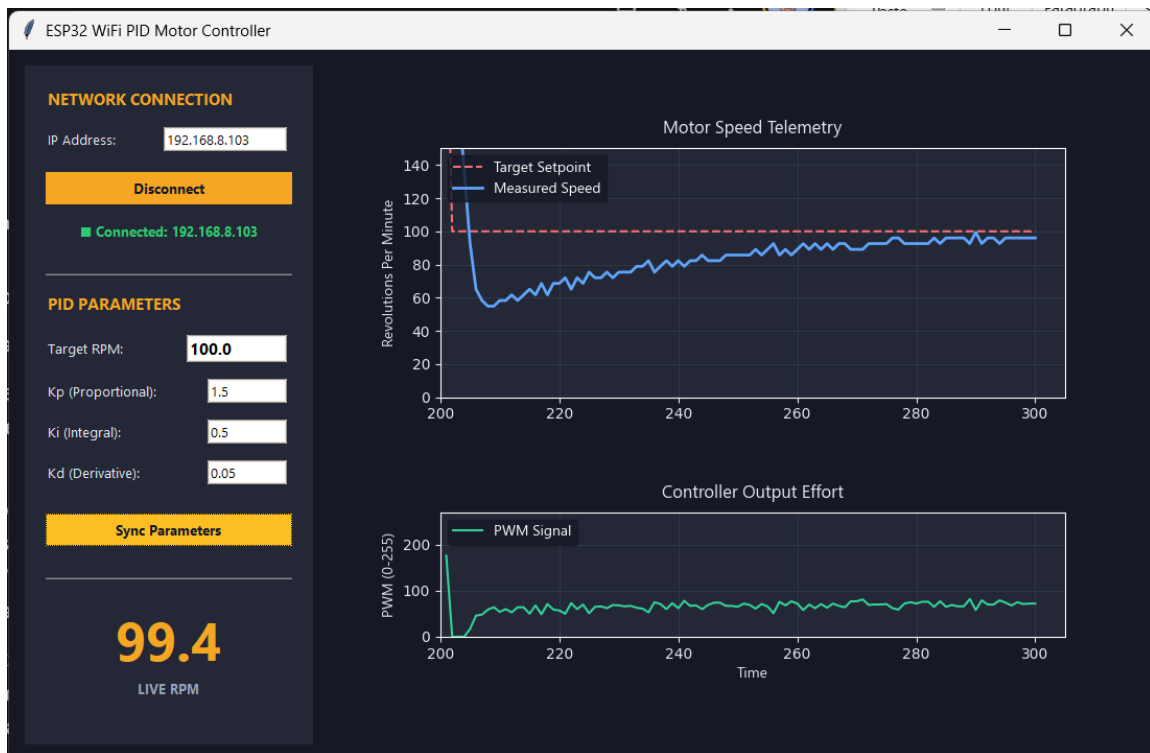


Figure 2: Real-time Python GUI displaying Motor Speed (RPM) and PWM Controller Output.

6. Discussion

The implementation of PID control demonstrates the effectiveness of DC motor speed regulation. Compared with open-loop control, the closed-loop system shows significant improvements in accuracy and stability. Tuning of PID parameters played a crucial role.

Tuning the PID parameters played a crucial role. Improper tuning initially led to oscillations and instability. By building a remote GUI, parameters could be adjusted seamlessly without having to constantly recompile and upload code to the ESP32. Through these iterative adjustments over WiFi, optimal values were achieved.

Limitations observed include:

- Occasional latency over the TCP WiFi connection affecting plotting smoothness.
- Delay due to sampling interval.

Despite these, the system achieved acceptable performance for practical applications.

7. Challenges Faced

- Difficulty in selecting initial appropriate PID gains
- Ensuring the GUI plot Thread did not block the socket reading thread in Python
- Preventing integral windup before implementing programmatic limits.
- Wiring and power supply issues common to motor inductive loads.

8. Improvements

- Apply filtering techniques (e.g., low-pass filter)
- Use advanced tuning methods (Ziegler–Nichols)
- Integrate real-time monitoring via serial plotter or IoT

9. Conclusion

This project successfully designed and implemented a PID-based DC motor speed control system using ESP32, L298N, and an encoder motor. The system demonstrated stable and accurate control under closed-loop operation. The PID controller effectively reduced errors, reduced oscillations, and improved system response. Combined with a custom-built WiFi monitoring interface, this project provided a strong practical understanding of control theory, wireless standardization, embedded systems, and real-time system design.

10. References

- 1) ESP32 Technical Reference Manual :
https://documentation.espressif.com/esp32_technical_reference_manual_en.pdf
- 2) L298N Datasheet : <https://www.st.com/resource/en/datasheet/l298.pdf>

Appendix A:

```
#include <WiFi.h>

#define ENB_PIN      25
#define ENC_A_PIN    18
#define ENC_B_PIN    19
#define SAMPLE_TIME_MS 50
#define PWM_FREQ     5000
#define PWM_BITS     8
#define LOG_INTERVAL_MS 100
#define MIN_PWM      0
#define MAX_PWM      255

// --- WiFi Settings ---
const char* ssid = "Router Name";
const char* password = "Router Password";
const int port = 8080;

WiFiServer server(port);
WiFiClient client;

float PPR = 350.0;

double Kp = 2.5;
double Ki = 1.5;
double Kd = 0.03;
double targetRPM = 250.0;

volatile long encoderCount = 0;
long lastEncoderCount = 0;
double measuredRPM = 0.0;
double pidOutput = 0.0;
double errorSum = 0.0;
double lastError = 0.0;
unsigned long lastTime = 0;
unsigned long lastLogTime = 0;

void IRAM_ATTR encoderISR_A() {
    if (digitalRead(ENC_B_PIN) == HIGH) {
        encoderCount++;
    } else {
        encoderCount--;
    }
}

void setMotorPWM(int pwmValue) {
    pwmValue = constrain(pwmValue, MIN_PWM, MAX_PWM);
    ledcWrite(ENB_PIN, pwmValue);
}

double calculateRPM() {
    long currentCount = encoderCount;
```

```

    long delta          = currentCount - lastEncoderCount;
    lastEncoderCount    = currentCount;
    double rpm = ((double)delta / PPR) / (SAMPLE_TIME_MS / 60000.0);
    return abs(rpm);
}

double computePID(double setpoint, double measured) {
    double dt          = SAMPLE_TIME_MS / 1000.0;
    double error        = setpoint - measured;

    errorSum            += error * dt;
    errorSum             = constrain(errorSum, -300.0, 300.0);

    double dError = (error - lastError) / dt;
    lastError      = error;

    double output = (Kp * error) + (Ki * errorSum) + (Kd * dError);
    return constrain(output, MIN_PWM, MAX_PWM);
}

void setup() {
    Serial.begin(115200);
    delay(500);

    // --- Connect to WiFi ---
    Serial.print("Connecting to WiFi: ");
    Serial.println(ssid);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\n--- WiFi Connected ---");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP()); // <-- NEED THIS IP FOR THE GUI

    server.begin();
    Serial.println("TCP Server started on port 8080.");

    ledcAttach(ENB_PIN, PWM_FREQ, PWM_BITS);
    ledcWrite(ENB_PIN, 0);

    pinMode(ENC_A_PIN, INPUT_PULLUP);
    pinMode(ENC_B_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(ENC_A_PIN), encoderISR_A, RISING);

    // Reset state
    encoderCount      = 0;
    lastEncoderCount  = 0;
    errorSum          = 0.0;
    lastError         = 0.0;
    lastTime          = millis();
    lastLogTime       = millis();
}

```

```

}

void loop() {
    unsigned long now = millis();

    // --- Handle WiFi Client Connection ---
    if (server.hasClient()) {
        if (!client || !client.connected()) {
            if (client) client.stop();
            client = server.available();
            Serial.println(">>> GUI Client Connected!");
        } else {
            server.available().stop(); // Reject additional clients if one is
already connected
        }
    }

    // --- PID Computation Loop ---
    if (now - lastTime >= SAMPLE_TIME_MS) {
        lastTime = now;
        measuredRPM = calculateRPM();
        pidOutput = computePID(targetRPM, measuredRPM);
        setMotorPWM((int)pidOutput);
    }

    // --- Sending Data to GUI ---
    if (now - lastLogTime >= LOG_INTERVAL_MS) {
        lastLogTime = now;
        String logMsg = "Setpoint:" + String(targetRPM, 1) +
            ",Measured:" + String(measuredRPM, 1) +
            ",PWM:" + String((int)pidOutput) + "\n";

        if (client && client.connected()) {
            client.print(logMsg); // Send over WiFi
        }
        Serial.print(logMsg); // Keep sending to Serial Monitor for debugging
    }

    // --- Receiving Data from GUI ---
    if (client && client.connected() && client.available()) {
        String input = client.readStringUntil('\n');
        input.trim();

        int firstComma = input.indexOf(',');
        if (firstComma > 0) {
            int secondComma = input.indexOf(',', firstComma + 1);
            int thirdComma = input.indexOf(',', secondComma + 1);

            if (secondComma > 0 && thirdComma > 0) {
                targetRPM = input.substring(0, firstComma).toDouble();
                Kp = input.substring(firstComma + 1, secondComma).toDouble();
                Ki = input.substring(secondComma + 1, thirdComma).toDouble();
                Kd = input.substring(thirdComma + 1).toDouble();
            }
        }
    }
}

```

```
errorSum = 0.0;
lastError = 0.0;

Serial.print(">>> Updated via WiFi -> RPM: ");
Serial.print(targetRPM);
Serial.print(" | Kp: "); Serial.print(Kp);
Serial.print(" | Ki: "); Serial.print(Ki);
Serial.print(" | Kd: "); Serial.println(Kd);
    }
  }
}
```

© Linuka Jayasinghe

Appendix B:

```
import tkinter as tk
from tkinter import ttk, messagebox
import socket
import threading
from collections import deque
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.animation as animation

class PIDControlApp:
    def __init__(self, root):
        self.root = root
        self.root.title("ESP32 WiFi PID Motor Controller")
        self.root.geometry("1050x650")

        # --- Data Storage ---
        self.max_points = 100
        self.x_data = deque(maxlen=self.max_points)
        self.setpoint_data = deque(maxlen=self.max_points)
        self.measured_data = deque(maxlen=self.max_points)
        self.pwm_data = deque(maxlen=self.max_points)
        self.time_counter = 0

        # --- Socket Configuration ---
        self.sock = None
        self.is_reading = False
        self.esp32_port = 8080

        self.setup_ui()
        self.setup_plot()

    def setup_ui(self):
        control_frame = ttk.Frame(self.root, padding="10")
        control_frame.pack(side=tk.LEFT, fill=tk.Y)

        # WiFi Connection setup
        ttk.Label(control_frame, text="ESP32 IP Address:").grid(row=0,
column=0, sticky=tk.W, pady=5)
        self.ent_ip = ttk.Entry(control_frame, width=15)
        self.ent_ip.insert(0, "192.168.8.103") # Your IP Address
        self.ent_ip.grid(row=0, column=1, pady=5)

        self.btn_connect = ttk.Button(control_frame, text="Connect via WiFi",
command=self.toggle_connection)
        self.btn_connect.grid(row=1, column=0, columnspan=2, pady=10,
sticky="ew")

        ttk.Separator(control_frame, orient='horizontal').grid(row=2,
column=0, columnspan=2, sticky='ew', pady=10)

        # PID Inputs
```



```

        ttk.Label(control_frame, text="Target RPM:").grid(row=3, column=0,
sticky=tk.W, pady=5)
        self.ent_rpm = ttk.Entry(control_frame, width=10)
        self.ent_rpm.insert(0, "100.0")
        self.ent_rpm.grid(row=3, column=1, pady=5)

        ttk.Label(control_frame, text="Kp:").grid(row=4, column=0,
sticky=tk.W, pady=5)
        self.ent_kp = ttk.Entry(control_frame, width=10)
        self.ent_kp.insert(0, "0.5") # Low Kp Value
        self.ent_kp.grid(row=4, column=1, pady=5)

        ttk.Label(control_frame, text="Ki:").grid(row=5, column=0,
sticky=tk.W, pady=5)
        self.ent_ki = ttk.Entry(control_frame, width=10)
        self.ent_ki.insert(0, "0.05") # Low Ki Value
        self.ent_ki.grid(row=5, column=1, pady=5)

        ttk.Label(control_frame, text="Kd:").grid(row=6, column=0,
sticky=tk.W, pady=5)
        self.ent_kd = ttk.Entry(control_frame, width=10)
        self.ent_kd.insert(0, "0.0")
        self.ent_kd.grid(row=6, column=1, pady=5)

        self.btn_send = ttk.Button(control_frame, text="Update PID
Parameters", command=self.send_parameters, state=tk.DISABLED)
        self.btn_send.grid(row=7, column=0, columnspan=2, pady=15,
sticky="ew")

        self.lbl_status = ttk.Label(control_frame, text="Disconnected",
foreground="red")
        self.lbl_status.grid(row=8, column=0, columnspan=2, pady=5)

        ttk.Separator(control_frame, orient='horizontal').grid(row=9,
column=0, columnspan=2, sticky='ew', pady=10)

        # --- Actual RPM Display Box ---
        self.lbl_actual_rpm = ttk.Label(control_frame, text="Actual RPM:
0.0", font=("Arial", 14, "bold"), foreground="blue")
        self.lbl_actual_rpm.grid(row=10, column=0, columnspan=2, pady=15)

    def setup_plot(self):
        plot_frame = ttk.Frame(self.root)
        plot_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        self.fig, (self.ax1, self.ax2) = plt.subplots(2, 1, figsize=(7, 5),
gridspec_kw={'height_ratios': [2, 1]})
        self.fig.tight_layout(pad=3.0)

        self.line_setpoint, = self.ax1.plot([], [], label='Setpoint RPM',
color='red', linestyle='--')
        self.line_measured, = self.ax1.plot([], [], label='Measured RPM',
color='blue')

```

```

        self.ax1.set_title("Motor Speed (RPM)")
        self.ax1.set_ylabel("RPM")
        self.ax1.legend(loc="upper left")
        self.ax1.grid(True)

        self.line_pwm, = self.ax2.plot([], [], label='PWM Output',
color='green')
        self.ax2.set_title("Controller Output (PWM)")
        self.ax2.set_ylabel("PWM (0-255)")
        self.ax2.set_xlabel("Time")
        self.ax2.legend(loc="upper left")
        self.ax2.grid(True)

        self.canvas = FigureCanvasTkAgg(self.fig, master=plot_frame)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

        self.ani = animation.FuncAnimation(self.fig, self.update_plot,
interval=100, cache_frame_data=False)

    def toggle_connection(self):
        if self.is_reading:
            self.is_reading = False
            if self.sock:
                self.sock.close()
                self.sock = None
            self.btn_connect.config(text="Connect via WiFi")
            self.btn_send.config(state=tk.DISABLED)
            self.lbl_status.config(text="Disconnected", foreground="red")
        else:
            ip_address = self.ent_ip.get().strip()
            if not ip_address:
                messagebox.showerror("Error", "Please enter the ESP32 IP
address.")
                return
            try:
                self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
                self.sock.settimeout(2.0)
                self.sock.connect((ip_address, self.esp32_port))
                self.sock.settimeout(None)

                self.is_reading = True
                self.btn_connect.config(text="Disconnect")
                self.btn_send.config(state=tk.NORMAL)
                self.lbl_status.config(text=f"Connected to {ip_address}",
foreground="green")

                threading.Thread(target=self.read_wifi_data,
daemon=True).start()
            except Exception as e:
                messagebox.showerror("Connection Error", f"Could not connect
to {ip_address}.\n{e}")

    def send_parameters(self):

```

```

if self.sock and self.is_reading:
    try:
        rpm = float(self.ent_rpm.get())
        kp = float(self.ent_kp.get())
        ki = float(self.ent_ki.get())
        kd = float(self.ent_kd.get())

        command = f"{rpm},{kp},{ki},{kd}\n"
        self.sock.sendall(command.encode('utf-8'))
    except ValueError:
        messagebox.showerror("Input Error", "Please enter valid
numbers.")

# The function to update the label on the UI thread
def update_rpm_display(self, rpm_val):
    self.lbl_actual_rpm.config(text=f"Actual RPM: {rpm_val:.1f}")

def read_wifi_data(self):
    buffer = ""
    while self.is_reading and self.sock:
        try:
            chunk = self.sock.recv(1024).decode('utf-8')
            if not chunk:
                break

            buffer += chunk
            while '\n' in buffer:
                line, buffer = buffer.split('\n', 1)
                line = line.strip()

                if "Setpoint:" in line and "Measured:" in line and "PWM:"
in line:
                    parts = line.split(',')
                    try:
                        sp = float(parts[0].split(':')[1])
                        meas = float(parts[1].split(':')[1])
                        pwm = float(parts[2].split(':')[1])

                        self.time_counter += 1
                        self.x_data.append(self.time_counter)
                        self.setpoint_data.append(sp)
                        self.measured_data.append(meas)
                        self.pwm_data.append(pwm)

                        # Update the actual RPM on the user interface
                        self.root.after(0, self.update_rpm_display, meas)

                    except (IndexError, ValueError):
                        pass
            except Exception as e:
                break

        self.root.after(0, self.force_disconnect)

```

```

def force_disconnect(self):
    if self.is_reading:
        self.toggle_connection()

def update_plot(self, frame):
    if not self.x_data:
        return

    self.line_setpoint.set_data(self.x_data, self.setpoint_data)
    self.line_measured.set_data(self.x_data, self.measured_data)
    self.ax1.set_xlim(max(0, self.time_counter - self.max_points),
self.time_counter + 5)

    current_sp = self.setpoint_data[-1] if self.setpoint_data else 300
    self.ax1.set_ylim(0, current_sp + 50)

    self.line_pwm.set_data(self.x_data, self.pwm_data)
    self.ax2.set_xlim(max(0, self.time_counter - self.max_points),
self.time_counter + 5)
    self.ax2.set_ylim(0, 270)

    return self.line_setpoint, self.line_measured, self.line_pwm

if __name__ == "__main__":
    root = tk.Tk()
    app = PIDControlApp(root)
    root.mainloop()

```